

Singly Linked List in Python

Complete 5-10 Minute Quick Revision Notes • Concepts, Operations, Pointer Wiring & Code

Chai aur DSA (Lecture #3)

Mentor: Hitesh Choudhary

Data Structures & Algorithms

Language: Python (Simple Hinglish)

1. Linked List Kya Hai & Iski Zaroorat Kyu Padi? (Array vs Linked List)

Linked List ek **Linear Data Structure** hai jisme elements (jinhe hum **Nodes** kehte hain) memory me contiguous (lagataar) store nahi hote, balki **Pointers/References** ke through ek doosre se jude hote hain.

⚠️ Array / Python List ki Limitations:

- **Contiguous Memory:** Elements ke liye lagataar memory chunk chahiye. Bada continuous block na mile toh fragmentation issue hota hai.
- **Costly Insert/Delete at Beginning:** Middle ya Start me insert/delete karne par baaki saare elements ko **shift** karna padta hai ($O(n)$ time).

✨ Linked List ke Superpowers:

- **Dynamic Memory Allocation:** Run-time par nodes randomly memory me kahin bhi ban sakti hain.
- **Fast Insert/Delete:** Start ya known position par sirf pointer link change karke $O(1)$ time me insertion/deletion ho jata hai.
- **Real-world Uses:** Spotify "Play Next Queue", Browser Back/Forward buttons, Graph Adjacency Lists.

2. Node Structure & Anatomy (Python Representation)

Har ek Node ke do main parts hote hain: **Data** (jo value store karni hai) aur **Next** (agla node ka reference/memory pointer).

[HEAD] → [Data: 10 | Next] → [Data: 20 | Next] → [Data: 30 | Next: None]

Node Class Implementation:

```
class Node:
    def __init__(self, data):
        self.data = data # Data store karega
        self.next = None # Next node ka pointer
```

SinglyLinkedList Wrapper Class:

```
class SinglyLinkedList:
    def __init__(self):
        self.head = None # First node ka pointer
```

3. Core Operations: Step-by-Step Code & Logic

A. Traversal & Display (Printing the List)

Traversal ke liye hum ek pointer temp ko head par rakhte hain aur tab tak aage badhte hain jab tak temp is None na ho jaye.

```
def display(self):
    temp = self.head
    while temp:
        print(temp.data, end=" -> ")
        temp = temp.next
    print("None")
```

B. Insertion Operations

1. Insert at Beginning ($\mathcal{O}(1)$):

1. Naya node banao: `new_node = Node(data)`
2. Naye node ka next purane head par point karo:
`new_node.next = self.head`
3. Head ko update karo: `self.head = new_node`

```
def insert_at_beginning(self, data):
    new_node = Node(data)
    new_node.next = self.head
    self.head = new_node
```

2. Insert at End ($\mathcal{O}(n)$ without Tail):

1. Agar list empty hai (head is None), toh `self.head = new_node`.
2. Warna last node tak travel karo (jiska `temp.next` is None).
3. Last node ke next me naya node attach karo: `temp.next = new_node`.

```
def insert_at_end(self, data):
    new_node = Node(data)
    if not self.head:
        self.head = new_node
        return
    temp = self.head
    while temp.next:
        temp = temp.next
    temp.next = new_node
```

C. Insert After a Specific Node Value

List traverse karo jab tak target value ka node na mil jaye. Phir naye node ka next target ke next par point karo, aur target ka next naye node par.

```
def insert_after(self, target_val, data):
    temp = self.head
    while temp and temp.data != target_val:
        temp = temp.next
    if not temp:
        print("Target node not found!")
        return
    new_node = Node(data)
    new_node.next = temp.next
    temp.next = new_node
```

D. Deletion Operations & Corner Cases (CRITICAL FOR INTERVIEWS)

- ⚠ **Important Corner Cases jo hamesha check karne hote hain:**
1. **Empty List:** Agar head is None ho toh return.
 2. **Delete First Node (Head):** Agar head hi target value hai, toh `self.head = self.head.next`.
 3. **Delete Middle / Last Node:** prev pointer maintain karo taaki target node bypass ho sake: `prev.next = current.next`.

```
def delete_node(self, val):
    if not self.head: return # Case 1: Empty list
    # Case 2: Target node head par hi hai
    if self.head.data == val:
        self.head = self.head.next
        return
    # Case 3: Middle ya End me se delete karna
    prev = None
    curr = self.head
    while curr and curr.data != val:
        prev = curr
        curr = curr.next
    if curr: # Element mil gaya, bypass pointer
        prev.next = curr.next
    else:
        print("Value not found in list!")
```

4. Time & Space Complexity Summary

Operation	Time Complexity	Space Complexity	Kyu? (Short Hinglish Reason)
Insert at Beginning	$\mathcal{O}(1)$	$\mathcal{O}(1)$	Directly head pointer update hota hai, koi iteration nahi.
Insert at End (no tail)	$\mathcal{O}(n)$	$\mathcal{O}(1)$	Puri list traverse karke last node tak jaana padta hai.
Insert After Node	$\mathcal{O}(n)$ (Search) + $\mathcal{O}(1)$ (Insert)	$\mathcal{O}(1)$	Search me time lagta hai, linking instant hoti hai.

Operation	Time Complexity	Space Complexity	Kyu? (Short Hinglish Reason)
Delete Node by Value	$\mathcal{O}(n)$	$\mathcal{O}(1)$	Pehle element search karna padta hai, phir prev . next update hota hai.
Search / Traversal	$\mathcal{O}(n)$	$\mathcal{O}(1)$	Index indexing possible nahi hai, ek ek karke travel karna hota hai.

5. ⚡ Quick Revision / Key Takeaways (5-Minute Scan)

- ♦ **Node = Data + Pointer:** Har node me data aur next pointer hota hai. Last node ka next hamesha None hota hai.
- ♦ **Head Pointer:** List ka start access point sirf head hota hai. Agar head kho gaya, toh puri list garbage collect ho jayegi!
- ♦ **No Contiguous Memory:** Array ki tarah continuous block ki zaroorat nahi hoti, memory runtime par dynamically allocate hoti hai.
- ♦ **Pointer Manipulation Order:** Hamesha pehle naye node ka link set karo (`new_node.next = curr.next`), uske baad pichle node ka link badlo. Galat order me head/node loss ho sakta hai!
- ♦ **Interview Golden Rule:** Code likhne se pehle diagrams draw karke Pointer wiring samjhao. Interviewer logic aur corner cases (Empty list, 1 node, Last node) dekhta hai.